FINANCE ADVISORS - SYSTEM DESIGN & ENGINEERING REPORT

Generated on August 3, 2026

Prepared by the Finance Advisors Engineering Team

Tab

1. Executive Summary

Finance Advisors is an educational market-condition platform that turns raw price, macro, and fundamentals data into readable technical signals, LLM-synthesized narrative reports, and downloadable PDFs. It integrates:

- Two independent Streamlit dashboards covering equities, gold/commodities, real estate proxies, and crypto
- A rule-based indicator layer (trend, momentum, valuation, breadth) computed with no LLM required
- An optional LLM synthesis step (Anthropic or OpenAI) that turns the computed scorecard into a written report
- A shared PDF export pattern (fpdf2) so any generated or saved report can be downloaded
- A separate eve agent scaffold (TypeScript, Vercel AI SDK, Claude Sonnet 5) as the seed of a future conversational/agentic layer

The system favors transparent, explainable signals over black-box scoring, so a user can always see *why* a lean is bullish, neutral, or bearish.

Educational project. Nothing in this system constitutes personalized financial advice.

2. Problem Statement

Retail-level market research is scattered across price charts, macro releases, valuation screens, and news - with no single place to see them together or to get a plain-English read on what they mean. Finance Advisors automates:

- Price/trend/momentum retrieval across equities, commodities, and real estate proxies
- Macro backdrop tracking (rates, inflation, employment, housing)
- Rule-based composite scoring with visible, itemized reasoning
- Optional LLM narrative synthesis over the computed scorecard plus recent headlines
- Exporting any generated report to a portable PDF

3. Project Objectives

- Provide consistent, transparent technical/fundamental signals with no black-box scoring
- Support two independent surfaces: a full multi-tab dashboard and a lightweight single-file exploratory tool
- Deliver clean, itemized reasoning for every buy/hold/sell-style read
- Enable PDF report generation and download from both dashboards
- Keep the two dashboards decoupled so either can evolve independently
- Lay the groundwork for a conversational agent layer via eve

4. High-Level System Overview

Finance Advisors consists of three independent parts that currently do not share a runtime or a network boundary:

market_insight_agent/ (Streamlit)

- Config-driven ticker/FRED-series universe
- Data layer (yfinance, FRED via pandas_datareader, NewsAPI/RSS)
- Indicator + scorecard layer
- LLM report synthesis (Anthropic/OpenAI) + Slack push
- CLI/scheduler entry point (main.py) for unattended runs
- PDF export for both the live-generated report and saved history

workspace/ (Streamlit)

- Self-contained single-file app with its own broader asset catalog
- Technical indicators (SMA/RSI/MACD/Bollinger) computed inline
- Rule-based composite score + optional OpenAI narrative
- PDF export of the on-screen analysis

agent/ (eve, TypeScript)

- defineAgent() scaffold wired to Claude Sonnet 5 via @ai-sdk/anthropic
- Not yet wired to either dashboard - currently a standalone seed for a future conversational layer (see Section 12)

5. High-Level Architecture Diagram

```
STREAMLIT DASHBOARD 1: market_insight_agent
 - Tabs: Equities, Gold, Real Estate, Macro, AI Insight, History
 - PDF download (session report + saved history)
v
CONFIG + DATA SOURCES
 - Ticker/FRED universe (config.py)
 - yfinance / pandas_datareader / NewsAPI / RSS (data_sources.py)
v
INDICATORS + SCORECARD
 - Trend, momentum, valuation, breadth (indicators.py)
 - Full scorecard assembly (scorecard.py)
v
LLM SYNTHESIS
 - Anthropic or OpenAI (config.LLM_PROVIDER)
 - Prompt template + system rules (prompt_template.py)
 - Report saved to disk + optional Slack push (report.py)
v
PDF EXPORT
 - markdown_to_pdf_bytes() (fpdf2)
 - Unicode-safe rendering for LLM-generated text

STREAMLIT DASHBOARD 2: workspace (independent)
 - Wider asset catalog, inline technical indicators
 - Rule-based composite score + optional OpenAI narrative
 - Same PDF export pattern, self-contained in one file

EVE AGENT (independent, TypeScript)
 - defineAgent() + Claude Sonnet 5 via Vercel AI SDK
 - Not yet connected to either dashboard
```

6. eve Agent Integration

The repository is scaffolded as an eve project (package.json scripts: eve dev, eve build, eve start). The agent definition is minimal today:

```
import { anthropic } from "@ai-sdk/anthropic";
import { defineAgent } from "eve";

export default defineAgent({
  model: anthropic("claude-sonnet-5"),
});
```

This establishes the model/runtime the eve framework will orchestrate, but no tools, retrieval, or task logic have been added yet - the two Streamlit dashboards currently operate entirely independently of this agent. Section 12 sketches how this layer could grow into the system's actual orchestration point.

7. Dashboard 1: Market Insight Agent

The primary, actively developed dashboard (market_insight_agent/app.py), backed by a small set of focused modules.

7.1 Configuration & Ticker Universe

config.py centralizes every tracked symbol and every environment-driven setting, loaded via python-dotenv:

```
EQUITY_TICKERS = {"S&P 500": "^GSPC", "Nasdaq 100": "^NDX", "Russell 2000": "^RUT"}
BREADTH_PROXY_TICKERS = {"Semiconductors (SOXX)": "SOXX", "Momentum factor (MTUM)": "MTUM", "Equal-we
  ight S&P (RSP)": "RSP"}
COMMODITY_TICKERS = {"Gold": "GC=F", "Gold ETF": "GLD"}
REAL_ESTATE_TICKERS = {"Homebuilders ETF (XHB)": "XHB", "REIT ETF (VNQ)": "VNQ"}
FRED_SERIES = {"30yr Mortgage Rate": "MORTGAGE30US", "Fed Funds Rate": "FEDFUNDS", ...}
```

LLM provider, model, Slack webhook, news API key, and output directory are all read from environment variables with sane defaults.

7.2 Data Sources & Caching

data_sources.py is a pure retrieval layer - no analysis, so data vendors can be swapped without touching indicator logic:

- fetch_price_history - daily OHLCV via yfinance
- fetch_fundamentals - trailing/forward P/E, 52-week high/low
- fetch_fred_series / fetch_all_macro - macro series via pandas_datareader, degrades gracefully (returns an empty series) if the FRED endpoint hiccups
- fetch_headlines - NewsAPI first (if NEWS_API_KEY is set), falls back to RSS feeds

In the Streamlit layer, all of these are wrapped with st.cache_data(ttl=900) so switching tabs doesn't re-hit yfinance/FRED on every rerun.

7.3 Indicators & Scorecard

indicators.py deliberately avoids black-box ML - every signal is a simple, inspectable rule:

- trend_signal - classifies uptrend/downtrend/mixed from price vs. 50/200-day MAs
- momentum_signal - % change over the last 21 trading days
- valuation_signal - trailing P/E vs. a configurable historical-average P/E
- macro_regime - 3-month rising/falling/flat direction per FRED series
- breadth_signal - % of sector/factor proxy ETFs trading above their 200-day MA

scorecard.py assembles all of the above per asset class into one structured dict (build_full_scorecard), which is what gets handed to the LLM synthesis step.

7.4 AI Report Generation

prompt_template.py builds the system + user prompt: the system prompt requires the model to state leans per asset class with 1-2 concrete supporting data points, flag the biggest risk to each lean, and explicitly disclaim that this is not financial advice. llm_client.py is a thin provider-agnostic wrapper - config.LLM_PROVIDER selects between an Anthropic (claude-sonnet-4-5 by default) or OpenAI call. report.py orchestrates the full pipeline: build scorecard -> synthesize -> save to reports/market_insight_{timestamp}.md -> optional Slack push. This same pipeline backs both the Streamlit "Generate" button and the CLI (main.py --once / --schedule "08:30" via APScheduler).

7.5 PDF Export

Both the live session report (AI Insight tab) and any previously saved report (History tab) can be downloaded as a PDF via markdown_to_pdf_bytes() - see Section 9 for the shared implementation details.

7.6 Streamlit UI Layout

Six tabs: Equities, Gold, Real Estate, Macro & Rates, AI Insight, History. The sidebar hosts a data-refresh button, the report generate/regenerate control (disabled until LLM_API_KEY is set), an optional Slack push button, and tracked-ticker/LLM-provider captions.

8. Dashboard 2: Standalone Workspace Agent

workspace/app.py is a single self-contained file with no shared modules - intentionally decoupled from market_insight_agent/ so it can be run, copied, or modified independently.

8.1 Asset Catalog & Analysis Pipeline

A broader, flatter catalog than Dashboard 1: Stocks & Indices (AAPL, MSFT, AMZN, GOOGL, NVDA, TSLA, S&P 500, Nasdaq 100, Dow Jones), Gold & Commodities, Real Estate REIT proxies, and Crypto (BTC, ETH, SOL). A user can also type any custom ticker. analyze_ticker() pulls 2 years of daily data via yfinance, computes indicators, fetches info and recent news, and returns one result dict per ticker.

8.2 Composite Scoring Model

compute_indicators() adds SMA20/50/200, RSI(14), MACD, and Bollinger Bands. build_score() combines these into a single -100..+100 composite score with an itemized reasons list (each reason tagged with its point contribution):

- Price vs. 200-day MA (±15), 50/200-day golden/death cross (±10)
- RSI oversold/overbought (±10), MACD vs. signal line (±10)
- 3-month momentum (±5), proximity to 52-week high (±5)
- Trailing P/E cheap/expensive vs. historical norms (±10)
- News headline sentiment via keyword scoring (±10, clipped)

recommendation_label() maps the score to Strong Buy / Buy / Hold / Sell / Strong Sell.

8.3 Narrative Generation

rule_based_narrative() is the default, deterministic text generator (always available, no API key needed). If the user opts in and supplies an OpenAI key, ai_narrative() asks gpt-4o-mini for a 4-6 sentence plain-English summary instead, falling back to the rule-based version on any failure.

8.4 PDF Export

build_report_markdown() assembles the on-screen analysis (metrics, recommendation, narrative, itemized signals, headlines) into one markdown document, which markdown_to_pdf_bytes() renders to a downloadable, timestamped PDF - see Section 9.

8.5 Streamlit UI Layout

Two tabs: Analysis (single-ticker deep dive with a 3-row Plotly chart - candlestick + moving averages, RSI, MACD) and Watchlist (comma-separated ticker list scored and

color-coded in a single table). The sidebar controls asset class, preset/custom ticker, chart lookback window, and the optional AI-narrative toggle.

9. PDF Generation Pipeline (Shared Design Pattern)

Both dashboards independently implement the same small pattern using fpdf2:

```
def markdown_to_pdf_bytes(text: str) -> bytes:
    text = _pdf_safe(text)
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_page()
    pdf.set_font("helvetica", size=11)
    for raw_line in text.splitlines() or [""]:
        # "# " -> H1, "## " -> H2, "**...**" -> bold line, blank -> spacer,
        # everything else -> wrapped paragraph text
        ...
    return bytes(pdf.output())
```

Key design points:

- No system dependencies. fpdf2 is pure Python - unlike WeasyPrint (which needs Cairo/Pango/GDK-pixbuf), it installs cleanly on Windows with no native toolchain.

- Unicode safety. Core PDF fonts (Helvetica/Times/Courier) only support Latin-1. LLM-generated text routinely contains em dashes, curly quotes, ellipses, and bullets that aren't in that character set. _pdf_safe() normalizes the common cases to ASCII and falls back to str.encode("latin-1", "replace") for anything else, so PDF generation can never crash on unexpected characters.

- Correct cursor advancement. fpdf2's multi_cell() defaults to leaving the cursor to the *right* of the printed block rather than advancing to a new line below it. Every call explicitly passes new_x="LMARGIN", new_y="NEXT" to get normal paragraph-stacking behavior (see Section 11).

- Placement matches the reading order of the page - in both the AI Insight and History tabs, the download button is rendered *before* the report content, so it's visible without scrolling past a long report first.

10. End-to-End Workflow

Dashboard 1 (market_insight_agent):
User opens dashboard -> Streamlit tabs render cached price/macro/fundamentals data through the indicator layer -> user clicks "Generate / Regenerate report" -> scorecard assembled -> LLM synthesizes narrative -> report saved to disk + shown in AI Insight tab -> user downloads current or historical report as PDF (optionally pushed to Slack).

Dashboard 2 (workspace):
User selects an asset (preset or custom ticker) -> price history + fundamentals + headlines fetched -> indicators computed -> composite score + recommendation derived -> narrative generated (rule-based or optional AI) -> user downloads the assembled analysis as a PDF.

11. Technical Challenges & Solutions

- fpdf2 cursor-advancement bug. multi_cell(0, h, text) without explicit positioning args left the cursor near the right margin after the first call, so the *second* multi_cell call computed an available width of ~0 and raised "Not enough horizontal space to render a single character." Root-caused by printing a minimal two-line reproduction and inspecting pdf.x/pdf.y after each call. Fixed by passing new_x="LMARGIN", new_y="NEXT" on every multi_cell invocation.

- FPDFUnicodeEncodingException on em dashes. LLM-generated report text contained a Unicode em dash (-), which the core Helvetica font (Latin-1 only) can't encode. Rather than bundling a Unicode TTF font (heavier, adds a binary asset to the repo), added a _pdf_safe() normalization pass mapping common "smart" typography to ASCII, with a encode("latin-1", "replace") safety net for anything else.

- Real API key committed to a template file. market_insight_agent/.env.example was found to contain a live-looking Anthropic API key instead of a placeholder. Confirmed via git ls-files that it had never actually been committed (.env* is gitignored), then replaced it with a placeholder and had the user revoke/rotate the real key as a precaution.

- Two independent dashboards, one repo. workspace/app.py and market_insight_agent/app.py were kept intentionally decoupled (no shared Python package) rather than introduced a premature shared library, since they serve different scopes (broad exploratory tool vs. full scheduled-report dashboard) and evolve on separate cadences.

12. Target Architecture (Future Direction)

The current system has no backend/API layer, no multi-agent orchestration, and no retrieval system - both dashboards call data sources and (optionally) an LLM directly from Streamlit callback code. If this system grows past a single-user local tool, a reasonable target architecture - deliberately mirroring patterns proven in comparable systems - would introduce:

12.1 A thin API layer. Move scorecard-building and report synthesis behind a small FastAPI (or eve-native) service with one endpoint (e.g. /analyze) so either dashboard - or a future non-Streamlit client - can request a scorecard/report without importing Python modules directly.

12.2 Orchestration via the eve agent. Today the agent/ scaffold (Section 6) is disconnected from both dashboards. It could become the actual orchestration point: classify the user's request (equities vs. macro vs. real estate vs. general), select which data sources and prompts to run, and enforce report-composition rules (e.g. always surface the biggest risk to a bullish lean) - the same role CrewAI's orchestrator plays in comparable systems, but implemented with eve's native agent/task primitives instead of introducing a second agent framework.

12.3 A retrieval layer for headlines/filings. fetch_headlines() currently returns whatever NewsAPI/RSS provides with no ranking. A hybrid retrieval step (embedding similarity + keyword/BM25 over a larger corpus of filings or news) would let the LLM synthesis step ground claims in specific retrieved evidence rather than a raw headline list - useful if the system expands beyond price/macro signals into filing-level analysis.

12.4 A shared report/PDF module. If a third surface is added, promote the markdown_to_pdf_bytes() / _pdf_safe() pair (currently duplicated in both dashboards by design, see Section 11) into a small shared package once there's a real second consumer beyond the two Streamlit apps - premature sharing today would cost more than the duplication it removes.

None of the above is scheduled work; this section exists to give future contributors a plausible next step rather than to commit to a roadmap.

13. Future Enhancements

- Persist watchlist and custom-ticker choices per user session or account
- Add a lightweight test suite around indicators.py (pure functions, easy to unit test)
- Historical backtesting of the composite scoring model against realized returns
- Batch/multi-ticker AI report generation in market_insight_agent
- Wire the eve agent into at least one dashboard as a conversational front-end

14. Conclusion

Finance Advisors favors transparent, rule-based signals and small, independently runnable pieces over a monolithic pipeline. The two dashboards intentionally do not share a runtime, which keeps each simple to reason about at the cost of some duplicated PDF/report logic - a tradeoff explicitly revisited in Section 12 rather than solved prematurely. The eve agent scaffold gives the project a clear path toward a conversational/orchestration layer without requiring a rewrite of either dashboard.

15. How This Report Was Generated

This report is written in Markdown (docs/SYSTEM_DESIGN_REPORT.md) as the single source of truth, and rendered to PDF by docs/generate_report_pdf.py:

1. The generator parses this Markdown file directly (headings, fenced code blocks, a special diagram fence for the colored architecture box, bold lines, and bullet lists) rather than duplicating the content in a second format.
2. Code blocks are rendered in a monospace font on a shaded background; the diagram fence is rendered as a bordered, tinted box to mirror the colored architecture boxes in the reference format.
3. A title page and a real, page-numbered table of contents are generated using fpdf2's built-in start_section() / insert_toc_placeholder() support.
4. fpdf2 was used instead of a WeasyPrint/HTML pipeline specifically because it has no system-level dependencies (no Cairo/Pango/GDK-pixbuf), so the report can be regenerated on any machine that already has the project's Python dependencies installed - no extra install step.
5. Run it with: python docs/generate_report_pdf.py, which writes docs/SYSTEM_DESIGN_REPORT.pdf.

16. References

- Streamlit. (2024). Streamlit Documentation. https://docs.streamlit.io/
- yfinance. (2024). yfinance Documentation. https://github.com/ranaroussi/yfinance
- FRED / pandas-datareader. (2024). https://pandas-datareader.readthedocs.io/
- Plotly. (2024). Plotly Python Graphing Library. https://plotly.com/python/
- Anthropic. (2024). Claude API Documentation. https://docs.anthropic.com/
- OpenAI. (2024). OpenAI API Documentation. https://platform.openai.com/docs
- fpdf2. (2024). fpdf2 Documentation. https://py-pdf.github.io/fpdf2/
- Vercel eve. (2024). eve Documentation. https://eve.dev/docs
- Vercel AI SDK. (2024). https://sdk.vercel.ai